

安全

评估 Docker 的安全性时，主要考虑三个方面:

- 由内核的命名空间和控制组机制提供的容器内在安全
- Docker 程序（特别是服务端）本身的抗攻击性
- 内核安全性的加强机制对容器安全性的影响

内核命名空间

Docker 容器和 LXC 容器很相似，所提供的安全特性也差不多。当用 `docker run` 启动一个容器时，在后台 Docker 为容器创建了一个独立的命名空间和控制组集合。

命名空间提供了最基础也是最直接的隔离，在容器中运行的进程不会被运行在主机上的进程和其它容器发现和作用。

每个容器都有自己独有的网络栈，意味着它们不能访问其他容器的 `sockets` 或接口。不过，如果主机系统上做了相应的设置，容器可以像跟主机交互一样的和其他容器交互。当指定公共端口或使用 `links` 来连接 2 个容器时，容器就可以相互通信了（可以根据配置来限制通信的策略）。

从网络架构的角度来看，所有的容器通过本地主机的网桥接口相互通信，就像物理机器通过物理交换机通信一样。

那么，内核中实现命名空间和私有网络的代码是否足够成熟？

内核命名空间从 2.6.15 版本（2008 年 7 月发布）之后被引入，数年间，这些机制的可靠性在诸多大型生产系统中被实践验证。

实际上，命名空间的想法和设计提出的时间要更早，最初是为了在内核中引入一种机制来实现 `OpenVZ` 的特性。而 `OpenVZ` 项目早在 2005 年就发布了，其设计和实现都已经十分成熟。

控制组

控制组是 Linux 容器机制的另外一个关键组件，负责实现资源的审计和限制。

它提供了很多有用的特性；以及确保各个容器可以公平地分享主机的内存、CPU、磁盘 IO 等资源；当然，更重要的是，控制组确保了当容器内的资源使用产生压力时不会连累主机系统。

尽管控制组不负责隔离容器之间相互访问、处理数据和进程，它在防止拒绝服务（DDOS）攻击方面是必不可少的。尤其是在多用户的平台（比如公有或私有的 PaaS）上，控制组十分重要。例如，当某些应用程序表现异常的时候，可以保证一致地正常运行和性能。

控制组机制始于 2006 年，内核从 2.6.24 版本开始被引入。

Docker服务端的防护

运行一个容器或应用程序的核心是通过 Docker 服务端。Docker 服务的运行目前需要 root 权限，因此其安全性十分关键。

首先，确保只有可信的用户才可以访问 Docker 服务。Docker 允许用户在主机和容器间共享文件夹，同时不需要限制容器的访问权限，这就容易让容器突破资源限制。例如，恶意用户启动容器的时候将主机的根目录 `/` 映射到容器的 `/host` 目录中，那么容器理论上就可以对主机的文件系统进行任意修改了。这听起来很疯狂？但是事实上几乎所有虚拟化系统都允许类似的资源共享，而没法禁止用户共享主机根文件系统到虚拟机系统。

这将会造成很严重的安全后果。因此，当提供容器创建服务时（例如通过一个 web 服务器），要更加注意进行参数的安全检查，防止恶意的用户用特定参数来创建一些破坏性的容器。

为了加强对服务端的保护，Docker 的 REST API（客户端用来跟服务端通信）在 0.5.2 之后使用本地的 Unix 套接字机制替代了原先绑定在 127.0.0.1 上的 TCP 套接字，因为后者容易遭受跨站脚本攻击。现在用户使用 Unix 权限检查来加强套接字的访问安全。

用户仍可以利用 HTTP 提供 REST API 访问。建议使用安全机制，确保只有可信的网络或 VPN，或证书保护机制（例如受保护的 stunnel 和 ssl 认证）下的访问可以进行。此外，还可以使用 HTTPS 和证书来加强保护。

最近改进的 Linux 命名空间机制将可以实现使用非 root 用户来运行全功能的容器。这将从根本上解决了容器和主机之间共享文件系统而引起的安全问题。

终极目标是改进 2 个重要的安全特性：

- 将容器的 root 用户映射到本地主机上的非 root 用户，减轻容器和主机之间因权限提升而引起的安全问题；
- 允许 Docker 服务端在非 root 权限下运行，利用安全可靠的子进程来代理执行需要特权权限的操作。这些子进程将只允许在限定范围内进行操作，例如仅仅负责虚拟网络设定或文件系统管理、配置操作等。

最后，建议采用专用的服务器来运行 Docker 和相关的管理服务（例如管理服务比如 ssh 监控和进程监控、管理工具 nrpe、collectd 等）。其它的业务服务都放到容器中去运行。

内核能力机制

能力机制（Capability）是 Linux 内核一个强大的特性，可以提供细粒度的权限访问控制。Linux 内核自 2.2 版本起就支持能力机制，它将权限划分为更加细粒度的操作能力，既可以作用在进程上，也可以作用在文件上。

例如，一个 Web 服务进程只需要绑定一个低于 1024 的端口的权限，并不需要 root 权限。那么它只需要被授权 `net_bind_service` 能力即可。此外，还有很多其他的类似能力来避免进程获取 root 权限。

默认情况下，Docker 启动的容器被严格限制只允许使用内核的一部分能力。

使用能力机制对加强 Docker 容器的安全有很多好处。通常，在服务器上会运行一堆需要特权权限的进程，包括有 ssh、cron、syslogd、硬件管理工具模块（例如负载模块）、网络配置工具等等。容器跟这些进程是不同的，因为几乎所有的特权进程都由容器以外的支持系统来进行管理。

- ssh 访问被主机上ssh服务来管理；
- cron 通常应该作为用户进程执行，权限交给使用它服务的应用来处理；
- 日志系统可由 Docker 或第三方服务管理；
- 硬件管理无关紧要，容器中也就不需执行 udevd 以及类似服务；
- 网络管理也都在主机上设置，除非特殊需求，容器不需要对网络进行配置。

从上面的例子可以看出，大部分情况下，容器并不需要“真正的”root 权限，容器只需要少数的能力即可。为了加强安全，容器可以禁用一些没必要的权限。

- 完全禁止任何 mount 操作；
- 禁止直接访问本地主机的套接字；
- 禁止访问一些文件系统的操作，比如创建新的设备、修改文件属性等；
- 禁止模块加载。

这样，就算攻击者在容器中取得了 root 权限，也不能获得本地主机的较高权限，能进行的破坏也有限。

默认情况下，Docker 采用白名单机制，禁用必需功能之外的其它权限。当然，用户也可以根据自身需求来为 Docker 容器启用额外的权限。

其它安全特性

除了能力机制之外，还可以利用一些现有的安全机制来增强使用 Docker 的安全性，例如 TOMOYO, AppArmor, SELinux, GRSEC 等。

Docker 当前默认只启用了能力机制。用户可以采用多种方案来加强 Docker 主机的安全，例如：

- 在内核中启用 GRSEC 和 PAX，这将增加很多编译和运行时的安全检查；通过地址随机化避免恶意探测等。并且，启用该特性不需要 Docker 进行任何配置。
- 使用一些有增强安全特性的容器模板，比如带 AppArmor 的模板和 Redhat 带 SELinux 策略的模板。这些模板提供了额外的安全特性。
- 用户可以自定义访问控制机制来定制安全策略。

跟其它添加到 Docker 容器的第三方工具一样（比如网络拓扑和文件系统共享），有很多类似的机制，在不改变 Docker 内核情况下就可以加固现有的容器。

总结

总体来看，Docker 容器还是十分安全的，特别是在容器内不使用 root 权限来运行进程的话。

另外，用户可以使用现有工具，比如 Apparmor, SELinux, GRSEC 来增强安全性；甚至自己在内核中实现更复杂的安全机制。